PROG IV:
Consumo de APIs

Profª. Tainá Isabela

API’S

API é o acrônimo para **Application Programming Interface** (Interface de Programação de Aplicações). É através de uma API que podemos conectar diversas partes de um sistema, e controlar o acesso a cada uma delas.

Por exemplo: o cenário onde todas as aplicações que precisam lidar com geolocalização necessitassem implementar suas próprias soluções para isso. O investimento financeiro e de tempo que cada sistema do tipo iria demandar é imenso, e se tornaria inviável.

E é justamente este um dos **problemas** que a utilização de uma API resolve.

API’S

Em termos simples, uma API atua como um **canal de comunicação**.

Ele permite que os desenvolvedores solicitem informações específicas, como horários de trens, do servidor web. O servidor, por sua vez, responde a essas solicitações fornecendo os dados necessários.

Esses dados podem então ser perfeitamente integrados ao aplicativo da web, permitindo que usuários acessem facilmente as informações do trem, façam reservas e planejem sua viagem, por exemplo.

API’S

Em outras palavras, quando você deseja interagir com um computador ou sistema para recuperar informações ou executar uma função, uma API permite que você comunique suas expectativas ao sistema para que ele possa entender sua **solicitação** e **responder** adequadamente.

Você pode pensar em uma API como um **mediador** entre usuários ou clientes e os recursos ou serviços da web que eles desejam acessar.

Para as empresas, ele também serve como uma solução para **compartilhar recursos e informações**, mantendo um certo nível de segurança, controle e autenticação, determinando quem está autorizado a acessar o quê.

BACK-END
FRONT-END
APIs

Benefícios do uso de APIs

1. **Facilidade de integração**: As APIs são incrivelmente versáteis e podem ser perfeitamente integradas em vários aplicativos de software.

2. **Esforço reduzido de desenvolvimento de software:** As APIs reduzem significativamente a carga de trabalho de desenvolvimento.

3. **Segurança aprimorada:** As APIs se destacam por fornecer um gateway de comunicação seguro. Eles garantem que diferentes componentes de um aplicativo possam interagir e trocar dados sem comprometer a integridade dos dados ou expor vulnerabilidades.

Padrão Rest

Rest é um **padrão arquitetural** que especifica uma série de princípios que devem ser seguidos para o desenvolvimento da comunicação de aplicações, como as API´s. Uma API Rest utiliza o protocolo **HTTP**, se beneficiando de todos os seus verbos:

GET (para retornar dados), **POST** (para armazenar dados, em uma base de dados ou storage, por exemplo), **DELETE** (para deletar dados) e outros.

A vantagem de se utilizar estes métodos, é que podemos lidar com o tráfego de dados na Web de forma mais simples, tratando-os de forma similar a como fazemos no próprio console com aplicações **CRUD**.

API RESTful - REST API

No campo de serviços web e design de API, os termos ‘RESTful API’ e ‘REST API’ são frequentemente usados de forma **intercambiável**, mas é importante distinguir entre os dois para uma compreensão mais clara de suas características e funcionalidades distintas.

O que é REST API?

Uma **API REST**, uma abreviatura de **Transferência de Estado Representativo Application Programming Interface** é um **protocolo de comunicação** que segue um conjunto definido de regras e princípios.

Facilita a interação entre diferentes aplicações e serviços. As APIs REST lidam com solicitações de clientes para **recursos específicos**, respondendo fornecendo informações de recursos relevantes em um formato facilmente compreensível para o cliente.

Além disso, permitem aos clientes **modificar itens do servidor e criar novos**, oferecendo versatilidade na interação de dados.

O que é uma API RESTful?

Uma API RESTful é uma Interface de Programação de Aplicativos (API) que segue os **princípios e restrições arquitetônicas do REST**.

Essas APIs interagem com serviços web RESTful e apresentam vantagens como **maior escalabilidade e maior vida útil**.

As APIs RESTful empregam solicitações HTTP padrão para acessar e manipular dados, fornecendo aos clientes quatro métodos principais: **GET** (recuperar um recurso), **POST** (criar um novo recurso), **PUT** (editar ou atualizar um recurso existente) e **DELETE** (remover um recurso).

Princípios da API REST

No domínio das APIs REST, a adesão a um conjunto de princípios fundamentais define a sua eficácia e versatilidade.

Essas 6 regras, frequentemente chamadas de "**Seis Regras das APIs REST**", formam a base da arquitetura RESTful, fornecendo uma base robusta para comunicação e interação eficientes entre clientes e servidores:

Princípios da API REST

1. **Separação Cliente-Servidor:** As APIs REST impõem uma separação estrita entre clientes e servidores, com interações iniciadas exclusivamente por clientes. Essa independência permite que o software cliente evolua independentemente dos servidores, promovendo flexibilidade e manutenibilidade.

2. **Interface uniforme:** Uma interface uniforme padroniza a comunicação entre clientes e APIs REST, facilitando a compatibilidade entre vários idiomas e aplicativos. Isso garante que os recursos sejam identificáveis e separados das representações enviadas ao cliente e que os recursos possam ser manipulados pelo cliente usando a representação recebida.

Princípios da API REST

3. **Stateless:** Interações sem estado garantem que cada ciclo de solicitação-resposta seja independente, sem depender de solicitações anteriores. Isso aumenta a escalabilidade e reduz o uso de memória do servidor, garantindo interações eficientes e independentes.

4. **Sistema em camadas:** As APIs REST acomodam camadas entre clientes e servidores, permitindo funções como segurança e tratamento de tráfego. Essas camadas adicionais não devem impactar as principais interações cliente-servidor, garantindo flexibilidade e resiliência do sistema.

Princípios da API REST

5. **Armazenável em cache:** As APIs REST oferecem suporte ao cache de dados, permitindo que os clientes armazenem recursos localmente para acesso mais rápido. As respostas especificam a capacidade de cache, a duração e as regras, otimizando o uso de recursos do servidor e reduzindo o tempo de carregamento da página.

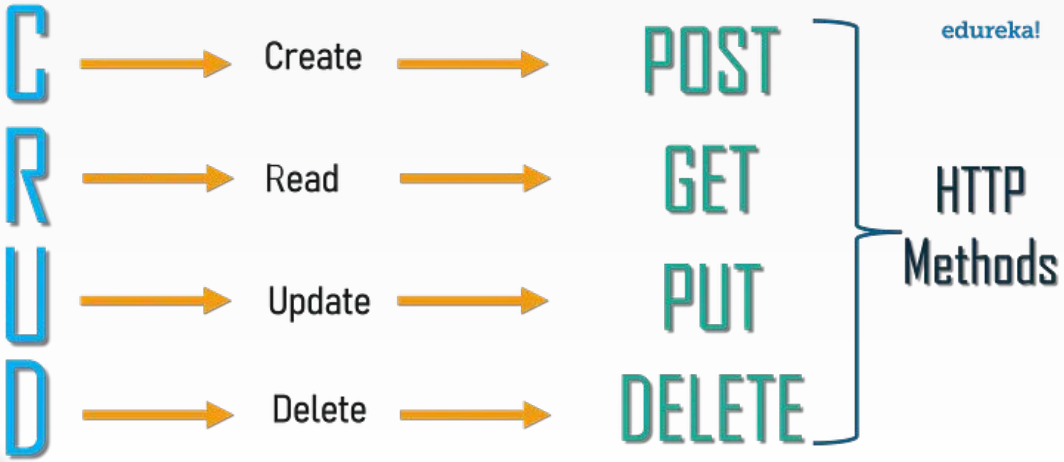
6. **Código sob demanda (opcional):** Como um recurso opcional, as APIs REST podem entregar código executável aos clientes, capacitando-os a executar código em seu próprio backend. Essa flexibilidade diferencia as APIs REST e as distingue de outros métodos de API, como SOAP.

Métodos HTTP em APIs RESTful

- **GET**: O método GET é usado para recuperar informações e recursos do servidor.
- **POST**: POST é empregado para criar novos recursos no servidor. Quando um cliente envia uma solicitação POST, ele envia dados ao servidor para criar um novo recurso.
- **PUT**: PUT é usado para atualizar ou substituir um recurso existente. Os clientes enviam uma solicitação PUT junto com a representação atualizada do recurso, e o servidor substitui o recurso existente pelos novos dados fornecidos na solicitação.
- **DELETE**: O método DELETE faz exatamente o que seu nome sugere. Ele exclui um recurso do servidor.

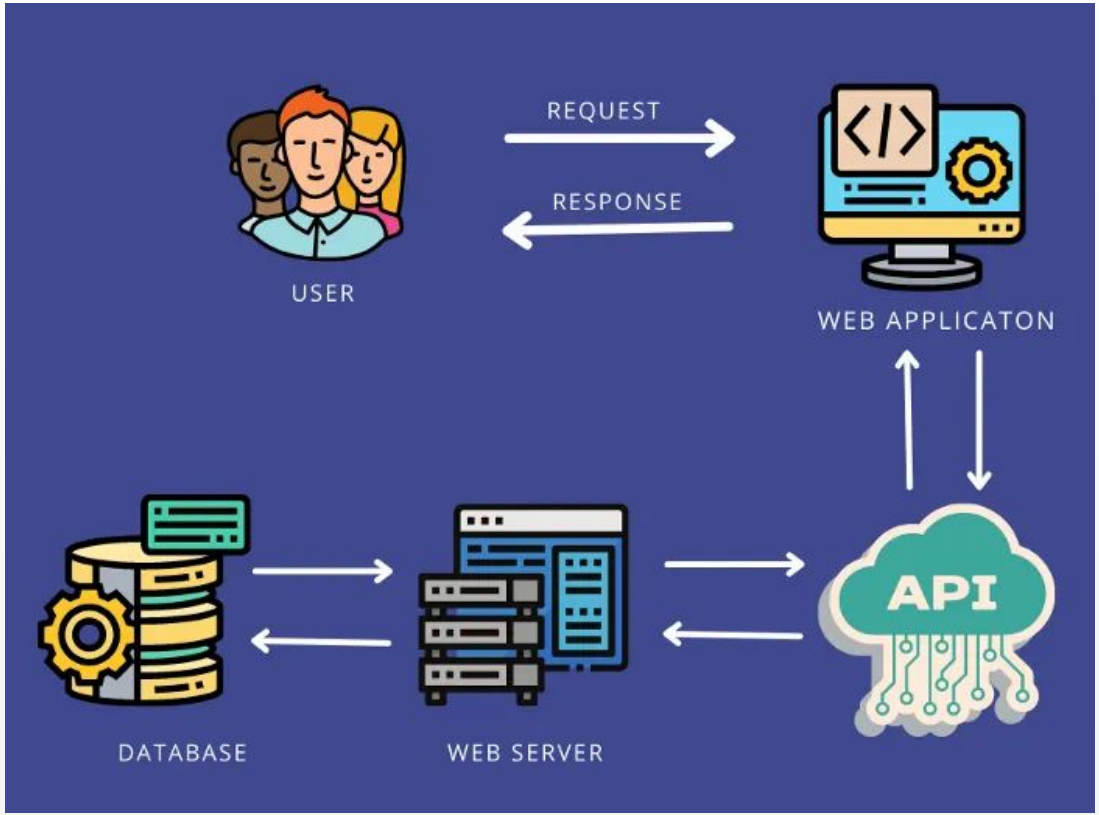
Métodos HTTP em APIs RESTful

Basicamente, os métodos compõem CRUD Operações (Criar, Ler, Atualizar, Excluir), tornando os endpoints da API intuitivos e fáceis de entender. Além disso, a adesão a esses métodos garante que a API permaneça consistente com os princípios REST, promovendo uma interface uniforme e previsível para os clientes.



Method	Purpose	Example Endpoint	Description
GET	Read data	/users	Fetch all users
POST	Create data	/users	Add a new user
PUT	Update data	/users/5	Replace user with ID 5
PATCH	Partial update	/users/5	Update part of user's data
DELETE	Delete data	/users/5	Remove user with ID 5

Code	Meaning	Scenario
200	OK	Request successful
201	Created	New resource added
400	Bad Request	Missing or invalid data
401	Unauthorized	Login or token required
403	Forbidden	You lack permission
404	Not Found	Resource doesn’t exist
500	Server Error	Server crashed


REQUEST
RESPONSE
USER
WEB APPLICATON
API
DATABASE
WEB SERVER

JSON

JSON (JavaScript Object Notation) é um formato leve de troca de dados, fácil de ler e escrever para humanos, e fácil de interpretar e gerar para máquinas.

É amplamente utilizado em APIs para enviar e receber dados, devido à sua estrutura simples e compatibilidade com diversas linguagens de programação.

JSON é baseado na sintaxe de objetos do JavaScript, o que o torna uma escolha natural para aplicações web, especialmente no front-end.

Sua facilidade de uso e formato humanamente legível ajudam a manter o código mais limpo e compreensível, tornando-o ideal para a troca de informações entre cliente e servidor.

JSON

O formato de sintaxe JSON é baseado em duas estruturas principais: Objetos e Matrizes. Um Objeto é uma coleção de pares nome/valor, e Array é uma lista ordenada de valores.

A sintaxe JSON é baseada em JavaScript Object Literals. Então, você verá que Objetos e Matrizes aqui têm a mesma sintaxe em JavaScript.

```
{
        "id": 4911,
        "primeiroNome":"John",
        "sobrenome":"Walter",
        "isStudent": verdadeiro
};
```

JSON

Algumas regras de formato de sintaxe para objetos em JSON:

- Um objeto é uma coleção de pares nome/valor.
- Começa com os colchetes encaracolados esquerdos ‘{’ e termina com os colchetes encaracolados direitos ‘}’
- Cada nome no Objeto é uma string entre aspas duplas e é seguido por dois pontos ‘:’ e depois pelo valor associado a esse nome.
- os pares nome/valor são separados por vírgula ‘,’

```
[
    "Bélgica", "Brasil", "Camarões", "Canadá", "Chile", "Dinamarca"
]
```

```
{
   "dados": [
      {
         "id": "X12",
         "de": {
            "nome": "John Kennedy", "id": "1458633"
         },
         "mensagem": "JSON é muito mais fácil e melhor que XML",
         "ações": [
            {
               "nome": "Comentário",
               "link": "http://www.facebook.com/X999/posts/Y999"
            },
            {
               "nome": "Como",
               "link": "http://www.facebook.com/X999/posts/Y999"
            }
         ],
         "tipo": "status",
         "tempo_criado": "2010-08-02T22:27:44+0000",
         "tempo_atualizado": "2010-08-02T22:27:44+0000"
      },
      {
         "id": "X45",
         "de": {
            "nome": "Anna Smith", "id": "12587521"
         },
         "mensagem": "A caminho de escrever meu primeiro trecho
JSON",
         "ações": [
            {
               "nome": "Comentário",
               "link": "http://www.facebook.com/X998/posts/Y998"
            },
            {
               "nome": "Como",
               "link": "http://www.facebook.com/X998/postagens/Y998"
            }
         ],
         "tipo": "status",
         "tempo_criado": "2010-08-02T25:27:44+0000",
         "tempo_atualizado": "2010-08-02T25:27:44+0000"
      }
   ]
}
```

X

```
<library>
  <book id="001">
    <title>The Pragmatic Programmer</title>
    <author>Andrew Hunt</author>
    <published>1999</published>
    <genre>Software Development</genre>
    <available>true</available>
  </book>

  <book id="002">
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <published>2008</published>
    <genre>Programming Practices</genre>
    <available>false</available>
  </book>

  <book id="003">
    <title>Atomic Habits</title>
    <author>James Clear</author>
    <published>2018</published>
    <genre>Self-Improvement</genre>
    <available>true</available>
  </book>
</library>
```

Frontend Dev
JSON
Backend Dev
Fullstack developer

Consumindo uma API - Pokeapi

```
<div class="controls">
  <input type="text" id="search" placeholder="Buscar Pokémon por nome ou ID">
  <button onclick="buscarPokemon()">Buscar</button>

  <select id="typeFilter" onchange="filtrarTipo()">
    <option value="">Todos os tipos</option>
    <option value="fire">Fire</option>
    <option value="water">Water</option>
    <option value="grass">Grass</option>
    <option value="electric">Electric</option>
    <option value="psychic">Psychic</option>
    <option value="rock">Rock</option>
    <option value="ground">Ground</option>
  </select>
</div>
```

Consumindo uma API - Pokeapi

```
<div class="grid" id="pokedex"></div>

<script>
  const pokedex = document.getElementById('pokedex');
  const loading = document.getElementById('loading');
  let listaPokemons = [];

  async function carregarPokemons() {
    listaPokemons = [];
    for (let i = 1; i <= 100; i++) {
      const res = await fetch(`https://pokeapi.co/api/v2/pokemon/${i}`);
      const data = await res.json();
      listaPokemons.push(data);
    }
    renderizar(listaPokemons);
  }
```

Consumindo uma API - Pokeapi

```
function renderizar(lista) {
    pokedex.innerHTML = '';
    lista.forEach(p => {
      const card = document.createElement('div');
      card.classList.add('card');
      card.innerHTML = `
        <img src="${p.sprites.front_default}">
        <h3>${p.name}</h3>
        <p>${p.types.map(t => t.type.name).join(', ')}</p>
      `;
      pokedex.appendChild(card);
    });
  }
```

Consumindo uma API - Pokeapi

```
async function buscarPokemon() {
    const termo = document.getElementById('search').value.toLowerCase();
    if (!termo) return renderizar(listaPokemons);

    try {
      const res = await fetch(`https://pokeapi.co/api/v2/pokemon/${termo}`);
      if (!res.ok) throw new Error();
      const data = await res.json();
      renderizar([data]);
    } catch {
      pokedex.innerHTML = '<p style="color:white; text-align:center;">Pokémon
não encontrado</p>';
    }
  }
```

Consumindo uma API - Pokeapi

```
function filtrarTipo() {
    const tipo = document.getElementById('typeFilter').value;
    if (!tipo) return renderizar(listaPokemons);
    const filtrados = listaPokemons.filter(p => p.types.some(t => t.type.name
=== tipo));
    renderizar(filtrados);
  }

  carregarPokemons();
</script>
```

Questionário